HAVENPOINT

Hotel Listing & Booking Platform

Table: HavenPoint Design Document Structure

Section No.	Section Title	Description
1	Title Page: HavenPoint	Contains project title, student details, course details, and submission information.
2	Introduction	Provides an overview of the HavenPoint system, its purpose, scope, and technologies used.
3	System Architecture	Describes the overall architecture of the system and explains how different components interact.
4	Functional Requirements	Lists the core functionalities that the system provides to users.
5	Use Case Diagram	Shows the interaction between users and the HavenPoint system.
6	Non-Functional Requirements	Describes performance, security, scalability, usability, and reliability requirements.
7	Database Design	Explains the database collections and schema used in the system.
8	ER Diagram	Represents relationships between entities such as Users, Listings, and Reviews.
9	Class Diagram	Shows the structure of system classes and their attributes and relationships.
10	Sequence Diagram	Illustrates how requests flow between system components over time.
12	Component Diagram	Shows how major software components interact within the system architecture.
13	Future Enhancements	Discusses potential improvements and additional features for future versions.
14	Conclusion	Summarizes the system design and overall functionality of the HavenPoint application.

Introduction

1.1 Purpose of the Document

This document presents the Software Design Document (SDD) for the HavenPoint web application. The purpose of this document is to describe the system architecture, database design, modules, and interactions between system components. It provides a detailed explanation of how the HavenPoint system is structured and implemented.

This document is intended for developers, evaluators, and stakeholders who want to understand the internal design of the system.

1.2 System Overview

HavenPoint is a web-based hotel listing and accommodation discovery platform that allows users to explore various properties, view detailed information, and share reviews about different locations.

The platform enables users to:

- Browse available accommodation listings
- View detailed information about properties
- Add new listings
- Edit or delete their listings
- Write reviews and ratings for properties

The system provides a user-friendly interface where travelers can easily discover places to stay while property owners can manage their listings efficiently.

1.3 Technologies Used

The HavenPoint system is developed using modern web technologies:

- Backend: Node.js
- Web Framework: Express.js
- Database: MongoDB
- Template Engine: EJS
- Image Storage: Cloudinary

These technologies work together to provide a scalable and efficient web application.

1.4 Scope of the System

The HavenPoint system focuses on providing a platform where users can explore and manage accommodation listings. The system handles user authentication, property management, and review management.

The current version of HavenPoint supports listing management and review functionality, while future enhancements may include booking systems, payment integration, and AI-based property recommendations.

System Overview & Architecture

HavenPoint is a web-based application designed to help users discover and manage accommodation listings. The system allows users to explore different properties, view detailed information about each listing, and share their experiences through reviews and ratings.

The application follows a **client–server architecture**, where users interact with the system through a web browser. The browser sends requests to the backend server, which processes the request and retrieves or updates data in the database.

The system is designed to be simple, scalable, and efficient so that users can easily browse listings and property owners can manage their accommodations.

3. System Architecture

The HavenPoint system follows the MVC (Model–View–Controller) architecture pattern, which separates the application into three main components to improve maintainability and scalability.

3.1 Model

The Model represents the **data layer** of the application. It defines the structure of the data stored in the database and handles interactions with the database.

In HavenPoint, the model layer manages collections such as:

- Users
- Listings
- Reviews
- Sessions

These collections are stored in the **MongoDB** database and are accessed using **Mongoose**.

3.2 View

The View represents the **presentation layer** of the application. It is responsible for displaying information to the user and providing an interface through which users can interact with the system.

In HavenPoint, the view layer is implemented using **EJS** templates that dynamically render data received from the backend.

3.3 Controller

The Controller acts as the **intermediate layer** between the Model and the View. It processes user requests, executes the required logic, and returns the appropriate response.

The controllers in HavenPoint are implemented using **Express.js** running on **Node.js**.

3.4 System Architecture Diagram

User (Web Browser)

|

Frontend (EJS Templates)

|

Express Server (Routes & Controllers)

|

Database Layer (MongoDB)

Explanation:

1. The user interacts with the application through a web browser.
2. The frontend sends requests to the backend server.
3. The backend processes the request using Express controllers.
4. The server retrieves or updates data stored in MongoDB.
5. The response is returned to the user interface.

Functional Requirements

Functional requirements describe the core features and operations that the HavenPoint system must perform. These requirements define how users interact with the system and what functionalities are provided by the application.

The main functional requirements of HavenPoint are listed below.

4.1 User Registration

The system allows new users to create an account by providing necessary details such as username, email address, and password. Once registered, users can log in to the platform and access additional functionalities.

4.2 User Authentication

The system provides a secure login mechanism where registered users can access their accounts using their credentials. Authentication ensures that only authorized users can perform actions such as adding listings or writing reviews.

4.3 View Listings

Users can browse different property listings available on the platform. Each listing displays important information such as title, description, location, price, and images.

4.4 Add Listing

Authenticated users can add new property listings to the platform. The system allows users to provide details such as property title, description, location, price, category, and images.

4.5 Edit Listing

Users who created a listing can update the details of that listing whenever required.

4.6 Delete Listing

Users can delete their own listings from the platform if the property is no longer available.

4.7 Add Review and Rating

Users can write reviews and provide ratings for different listings. Reviews help other users understand the quality and experience of a particular property.

Use Case Diagram

5.1 Overview

The Use Case Diagram represents the interaction between external users and the HavenPoint system. It illustrates the different functionalities that the system provides and how users interact with these functionalities.

In the HavenPoint platform, the primary actor is the **User**, who interacts with the system to perform operations such as browsing property listings, creating new listings, managing existing listings, and writing reviews. The use case diagram helps in understanding the system behavior from the user's perspective.

The diagram visually represents the system boundary and the various actions that can be performed within the system.

5.2 Primary Actor

User

A user represents any person who interacts with the HavenPoint platform. Users can register, log in, explore property listings, create new listings, manage their own listings, and share reviews about properties.

5.3 Use Case Description Table

Use Case	Description
Register	Allows a new user to create an account on the HavenPoint platform.
Login	Allows registered users to authenticate and access their accounts.
View Listings	Users can browse available accommodation listings on the platform.
ViewListing Details	Users can view detailed information about a specific property including description, price, and location.
Add Listing	Authenticated users can create a new listing by providing property details.
Edit Listing	Listing owners can modify the details of their listings.
Delete Listing	Listing owners can remove their listings from the platform.
Add Review	Users can write reviews and provide ratings for a property.
View Reviews	Users can read reviews written by other users for a listing.

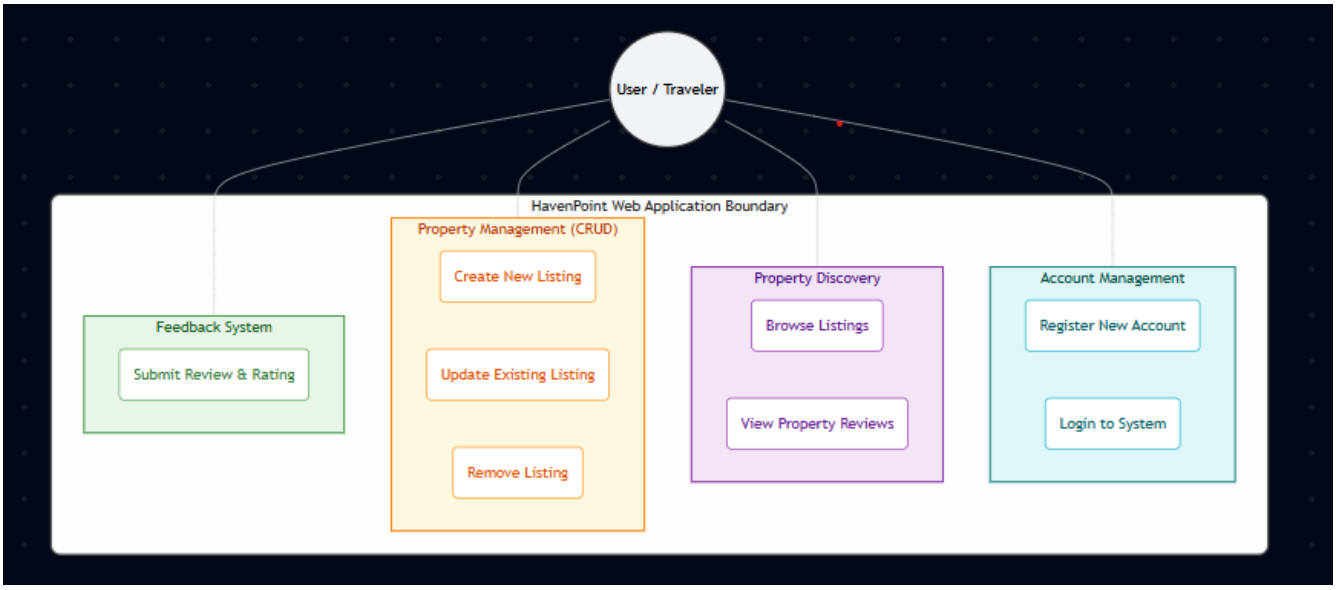


Fig : Use Case Diagram

Non-Functional Requirements

Non-functional requirements define the quality attributes and performance characteristics of the HavenPoint system. These requirements describe how the system should operate rather than what functionalities it provides. They ensure that the system is reliable, efficient, secure, and easy to use.

6.1 Performance

The system should provide fast and efficient responses to user requests. Pages such as property listings and listing details should load quickly to ensure a smooth user experience. The system should be capable of handling multiple users accessing the platform simultaneously without performance degradation.

6.2 Security

Security is an important aspect of the HavenPoint system. User authentication ensures that only authorized users can perform actions such as creating, editing, or deleting listings. Sensitive user information such as passwords is securely stored using hashing mechanisms.

6.3 Scalability

The system should be designed in such a way that it can support an increasing number of users and listings over time. The use of **MongoDB** as the database and **Node.js** as the backend allows the system to scale efficiently as the platform grows.

6.4 Usability

The HavenPoint platform should provide a simple and intuitive user interface that allows users to easily browse listings, view details, and manage their properties. The navigation should be clear and user-friendly so that even first-time users can interact with the system without difficulty.

6.5 Reliability

The system should function consistently and reliably. It should handle user requests correctly and prevent data loss during operations such as adding or updating listings.

6.6 Maintainability

The HavenPoint system follows a modular architecture which makes it easier to maintain and update the application. Developers can modify or extend specific modules without affecting the entire system.

Database Design

The HavenPoint system uses a NoSQL database to store and manage application data. The database stores information related to users, property listings, reviews, and user sessions.

The system uses **MongoDB** as the database because it provides flexibility, scalability, and efficient storage of structured and semi-structured data. The database structure is organized into multiple collections where each collection stores documents representing different entities of the system.

The major collections used in the HavenPoint system are:

- Users
- Listings
- Reviews
- Sessions

7.1 Listings Collection Schema

The Listings collection stores information about the properties listed on the platform.

Field	Type	Description
_id	ObjectId	Unique identifier for each listing
title	String	Title of the property
description	String	Description of the property
image	Object	Contains image URL and filename
price	Number	Price of the property
location	String	City or location of the property
country	String	Country where property is located
reviews	Array	Stores references to review IDs
owner	ObjectId	Reference to the user who created the listing
geometry	Object	Stores geographical coordinates
category	String	Category of the listing

7.2 Users Collection Schema

The Users collection stores account information for registered users.

Field	Type	Description
_id	ObjectId	Unique user identifier
email	String	User email address
username	String	Unique username
salt	String	Salt used for password hashing
hash	String	Hashed password

Field	Type	Description
__v	Number	Version key used by Mongoose

7.3 Reviews Collection Schema

The Reviews collection stores reviews written by users for listings.

Field	Type	Description
_id	ObjectId	Unique review identifier
comment	String	Review message
rating	Number	Rating provided by user
createdAt	Date	Time when review was created
author	ObjectId	Reference to the user who wrote the review

7.4 Sessions Collection Schema

The Sessions collection stores session information used for maintaining user login sessions.

Field	**Type**	**Description**
_id	String	Unique session identifier
expires	Date	Session expiration time
lastModified	Date	Last session update time
session	String	Encoded session data

Entity Relationship Diagram

The Entity Relationship (ER) Diagram represents the relationships between different entities in the HavenPoint system. It provides a clear visualization of how data is organized and how different components of the database interact with each other.

In the HavenPoint application, the primary entities include **Users**, **Listings**, and **Reviews**. These entities are connected through relationships that define how users create listings and interact with them through reviews.

The ER diagram helps developers understand how the database is structured and how information flows between different parts of the system.

8.1 Entities in the System

The main entities involved in the HavenPoint system are:

User

The User entity represents individuals who use the platform. Users can register, log in, create listings, and write reviews for properties.

Important attributes include:

- User ID
- Username
- Email
- Password (stored securely using hashing)

Listing

The Listing entity represents the properties available on the platform. Each listing contains information such as title, description, price, location, and images.

Important attributes include:

- Listing ID
- Title
- Description
- Price
- Location
- Country
- Category

Review

The Review entity represents feedback provided by users about a specific listing. Each review includes a rating and a comment describing the user's experience.

Important attributes include:

- ❿ Review ID
- ❿ Comment
- ❿ Rating
- ❿ Created Date

8.2 Relationships Between Entities

The relationships between the entities in the HavenPoint system are as follows:

1. **User – Listing Relationship**

A user can create multiple listings on the platform. However, each listing belongs to only one user.

Relationship Type:

One-to-Many (1:N)

Meaning:

One User → Many Listings

2. **User – Review Relationship**

A user can write multiple reviews for different listings.

Relationship Type:

One-to-Many (1:N)

Meaning:

One User → Many Reviews

3. **Listing – Review Relationship**

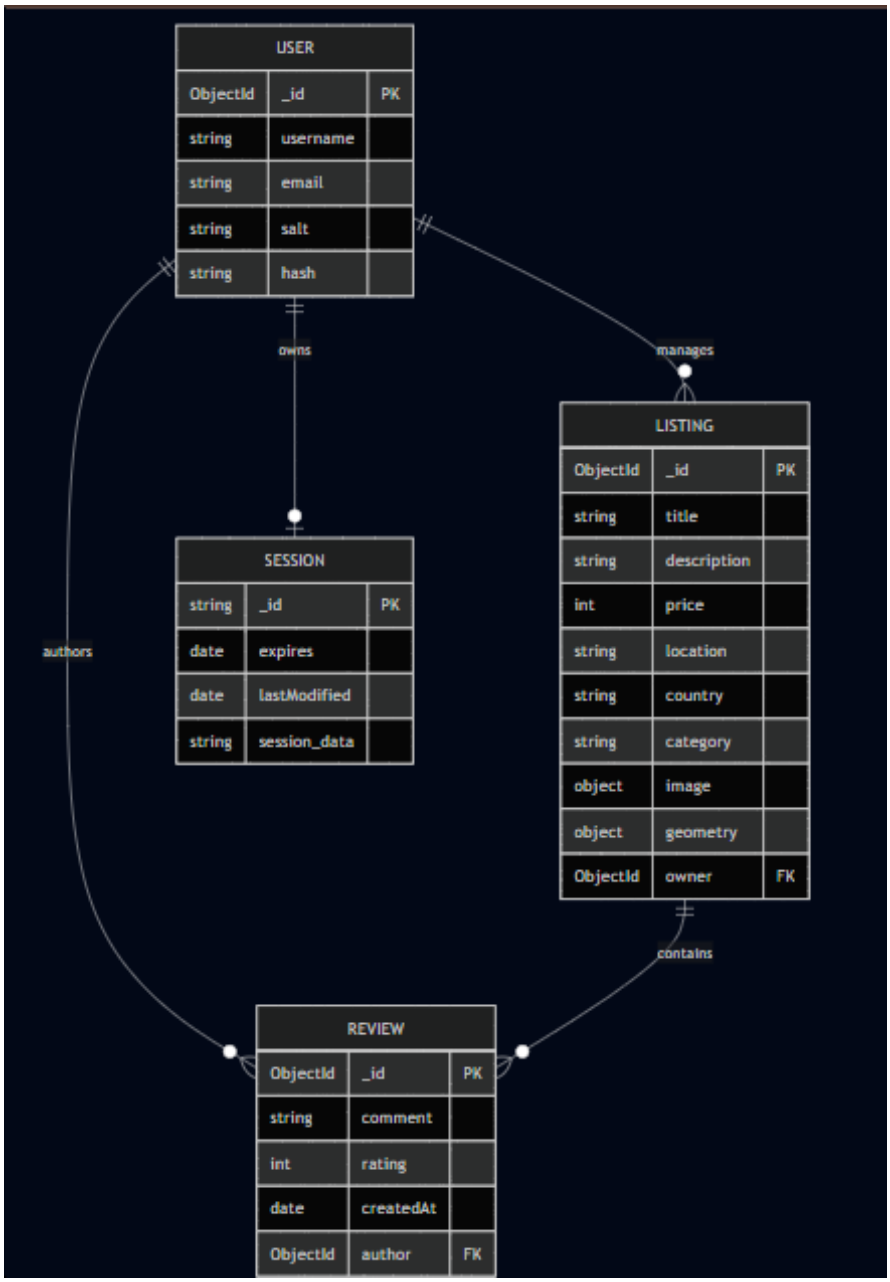
A listing can have multiple reviews written by different users.

Relationship Type:

One-to-Many (1:N)

Meaning:

One Listing → Many Reviews


USER
ObjectId _id PK
string username
string email
string salt
string hash
owns
manages
LISTING
ObjectId _id PK
string title
string description
int price
string location
string country
string category
object image
object geometry
ObjectId owner FK
SESSION
string _id PK
date expires
date lastModified
string session_data
authors
contains
REVIEW
ObjectId _id PK
string comment
int rating
date createdAt
ObjectId author FK

Sequence Diagram

A Sequence Diagram represents the flow of interactions between different components of the system in a particular order. It shows how objects communicate with each other and how requests are processed within the system.

In the HavenPoint system, sequence diagrams help illustrate how user actions are processed by the frontend, backend server, and database. This provides a clear understanding of the internal workflow of the application.

The main components involved in the interaction are:

- User (Browser)
- Frontend Interface
- Server
- Database

The backend server is implemented using **Node.js** and **Express.js**, while the data is stored in **MongoDB**.

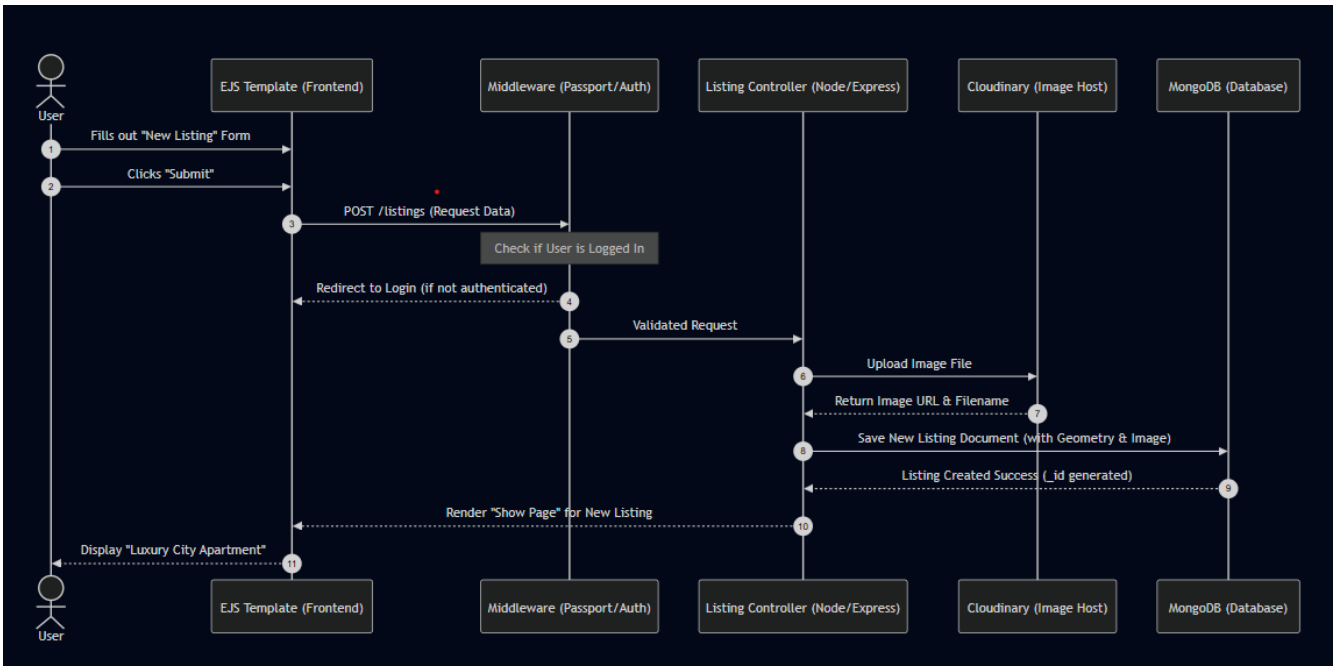


Fig : Sequence Diagram

Component Diagram

A Component Diagram represents the high-level structure of the HavenPoint system by showing how different software components interact with each other. It illustrates the organization of system modules and their dependencies within the application.

The HavenPoint application follows a **three-tier architecture** consisting of the Client Tier, Application Tier, and Data & Services Tier. Each tier is responsible for specific operations that together ensure smooth functioning of the system.

The component diagram helps developers understand how different parts of the system communicate and how data flows between them.

9.1 Client Tier

The Client Tier represents the user interface through which users interact with the HavenPoint system.

This tier includes:

- **Web Browser** – The user accesses the HavenPoint application using a browser.
- **EJS Templates** – These templates dynamically render the user interface and display data retrieved from the backend server.

The browser sends HTTP requests to the backend server whenever a user performs actions such as viewing listings or adding reviews.

9.2 Application Tier

The Application Tier contains the core logic of the HavenPoint system. It processes requests received from the client and performs operations related to listings, reviews, and authentication.

Major components in this tier include:

Listing Controller

This component manages operations related to property listings, including creating, updating, deleting, and displaying listings.

Review Controller

The Review Controller handles operations related to user reviews and ratings for listings.

Mongoose Models

The models define the structure of the data stored in the database and act as an interface between the application and the database.

9.3 Data and Services Tier

The Data and Services Tier manages persistent data storage and external services used by the application.

Key components include:

Database Service

The HavenPoint system stores data such as users, listings, and reviews in **MongoDB Atlas**, a cloud-based NoSQL database.

Image Storage Service

Images uploaded for listings are stored using **Cloudinary**, which provides efficient cloud-based image storage and management.

Geolocation Service

The system uses **Mapbox** to obtain geographical coordinates and map-based location services for listings.

9.4 Working of the Component Diagram

The interaction between components occurs in the following sequence:

1. The user interacts with the system through a web browser.
2. The browser sends HTTP requests to the server.
3. The request passes through authentication middleware.
4. Controllers process the request and perform the required business logic.
5. Mongoose models interact with the database to retrieve or store data.
6. External services such as Cloudinary and Mapbox provide additional functionalities.
7. The processed data is returned to the frontend and displayed to the user.

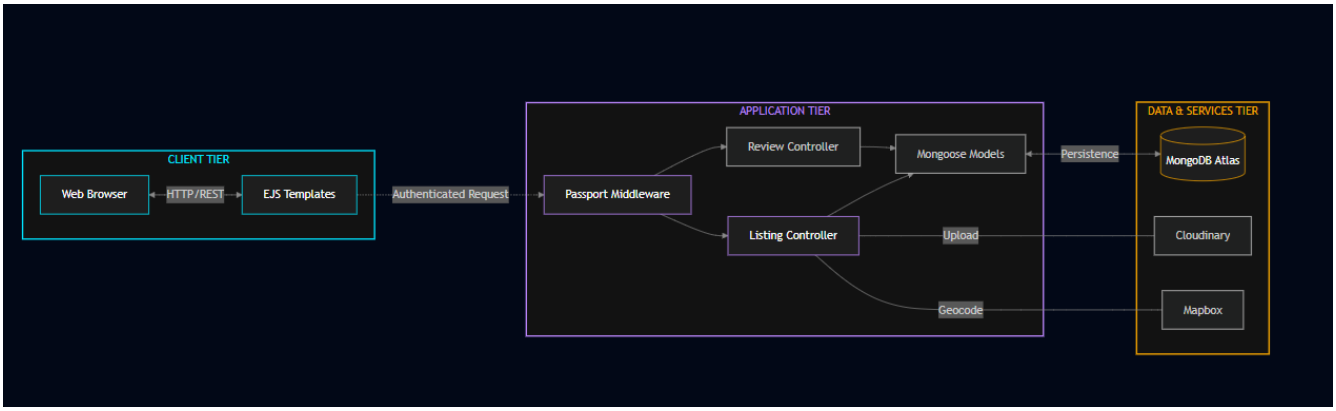


Fig : Component Diagram

Future Enhancements

Although HavenPoint currently provides essential functionalities for browsing property listings and writing reviews, there are several improvements that can be implemented in future versions to enhance the system's capabilities and user experience.

Some possible future enhancements include:

10.1 Online Booking System

A complete booking system can be integrated into HavenPoint so that users can directly reserve accommodations through the platform. This feature would allow users to select dates, check availability, and confirm bookings in real time.

10.2 Payment Integration

Secure online payment systems can be integrated to allow users to pay for bookings directly through the platform. Payment gateways would enable smooth and secure transactions between users and property owners.

10.3 AI-Based Property Recommendation

Artificial Intelligence techniques can be implemented to recommend properties to users based on their browsing behavior, previous searches, and preferences. This would help users discover suitable accommodations more efficiently.

10.4 Map-Based Property Search

The system can include an advanced map-based interface where users can explore listings visually based on their geographical location. This feature would improve the browsing experience by allowing users to view nearby accommodations on an interactive map.

10.5 User Ratings and Ranking System

A more advanced rating and ranking system can be implemented to highlight top-rated properties and trusted hosts. This would help users identify the most reliable accommodations.

Conclusion

The HavenPoint system provides a web-based platform that allows users to explore accommodation listings and share their experiences through reviews and ratings. The system is designed using a modular architecture that separates the application into different layers for better maintainability and scalability.

The use of modern technologies such as **Node.js**, **Express.js**, and **MongoDB** enables the system to efficiently manage data and handle user interactions.

Through the implementation of structured design principles and clear system architecture, HavenPoint provides a scalable and user-friendly solution for discovering accommodation listings. The platform can be further expanded with additional features such as booking systems, payment integration, and intelligent recommendation mechanisms in future versions.